



PROGRAMAÇÃO



Escola Naval



António Gonçalves

Agenda — Funções II

Âmbito das variáveis

Variáveis locais, globais e parâmetros



Distinguir variáveis locais, globais e parâmetros, e perceber como são usadas nas funções.

Composição de funções

Utilizar o resultado de uma função noutra



Utilizar o resultado de uma função como argumento noutra função, permitindo compor e reutilizar funcionalidade.

Organização do programa

Função principal e módulos



Organizar o programa em módulos, separando as funções da função principal, para tornar o código mais claro e reutilizável.

Reorganização e testes

Funções pequenas, assert e casos-limite



Reorganizar o programa em funções pequenas e testar o seu comportamento usando assert e casos-limite, para garantir que os resultados se mantêm.



Objetivo: Reorganizar um programa e testar as suas funções.

Revisão: parâmetros e return

Código em Python

```
def quadrado(n):  
    return n * n
```

```
valor = 4  
resultado = quadrado(valor)  
print(resultado)
```

Saída no ecrã

16

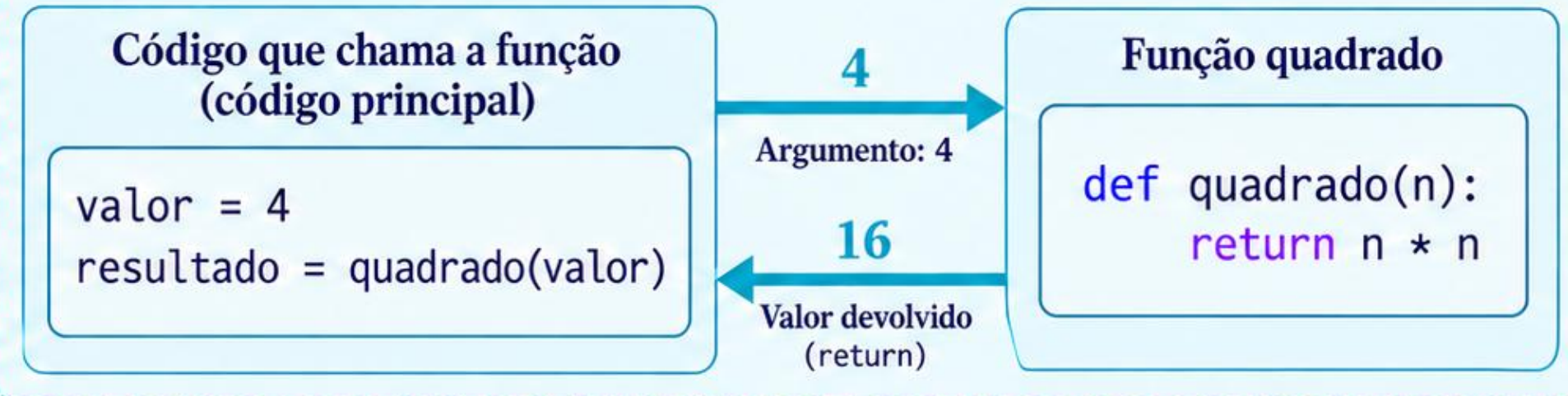
Parâmetro:

é o nome (variável) definido na função para receber um valor.

Valor devolvido (return):

A expressão de **return** é avaliada e o seu valor é devolvido ao código que chamou a função.

Funcionamento da chamada de função



O que acontece:

A variável **valor**, que vale 4, é passada como argumento para o parâmetro **n**. A função calcula **n * n**, devolve 16 e esse valor é atribuído à variável **resultado**.

E se chamarmos quadrado(0)?

A função devolve 0, porque $0 * 0$ é igual a 0.

VARIÁVEIS LOCAIS E ÂMBITO

Âmbito: região do programa onde um nome pode ser acessado.

```
def calcular():  
    resultado = 25  
    print(resultado)
```

```
calcular()  
print(resultado) # erro
```

Saída:

25

NameError: name 'resultado' is not defined

Função calcular()

variáveis locais

resultado = 25

print(resultado)

Fora da função (âmbito global)

resultado *(não definido)*

O nome resultado existe apenas dentro desta função (âmbito local).

Parâmetros e variáveis locais

Os parâmetros e os nomes atribuídos dentro de uma função são locais, nestes exemplos.

Funções diferentes

Funções diferentes podem ter nomes locais iguais sem os compartilharem entre si.

Aqui o nome resultado não existe, por isso ocorre um erro.

Cada função tem o seu próprio âmbito local.

Nomes locais iguais em funções diferentes referem-se a variáveis distintas.

ATRIBUIÇÃO A UM PARÂMETRO

Código em Python:

```
def alterar(numero):  
    numero = numero + 10  
    print("Dentro:", numero)  
  
valor = 5  
alterar(valor)  
print("Fora:", valor)
```

Saída:

Dentro: 15
Fora: 5

O que acontece:

1

Antes da chamada da função.

Âmbito global
(código principal)

valor

5

5

objeto inteiro

A função é chamada com o valor de valor, que é 5. O parâmetro numero recebe este valor.

2

Dentro da função, após a atribuição.

Âmbito global
(código principal)

valor

5

5

objeto inteiro

Dentro da função, numero é reatribuído com `numero + 10`, passando a 15. O nome valor no código principal mantém o valor 5.

Âmbito local
(função alterar)

numero

15

15

objeto inteiro

A reatribuição do nome local não altera o nome no código que chamou a função.

Neste exemplo, os valores são inteiros.

O mesmo nome em âmbitos diferentes

Código em Python

```
x = 10

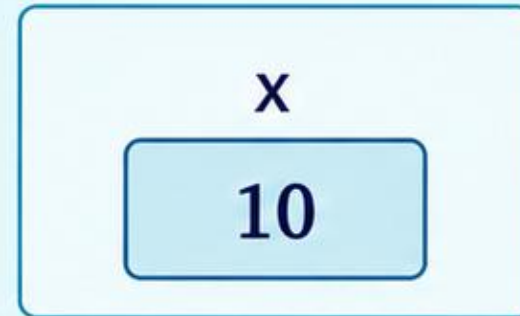
def mostrar():
    x = 3
    print(x)

mostrar()
print(x)
```

Saída no ecrã

```
3
10
```

Âmbito global



Variável global
(existe fora da função)

Âmbito local (dentro da função)



Variável local
(existe apenas na função)

É possível usar o mesmo nome em âmbitos diferentes sem que sejam a mesma variável.

A atribuição a `x` dentro da função não altera o valor da variável global.

Como funciona

Quando a função `mostrar()` é chamada, é criada uma variável local `x`, independente da variável global `x`. A instrução `print(x)` dentro da função mostra o valor local (3). Depois da função terminar, a variável global `x` mantém o seu valor e a segunda instrução `print(x)` mostra 10.

Boa prática

Os nomes podem coincidir, mas representam variáveis diferentes. É recomendável usar nomes claros e distintos para evitar confusões.

Globais e constantes

1 Variável global: alteração

Uma variável global definida no nível do módulo pode ser lida dentro de funções. É necessário usar a palavra-chave **global** para lhe atribuir um novo valor na função.

```
contador = 0
```

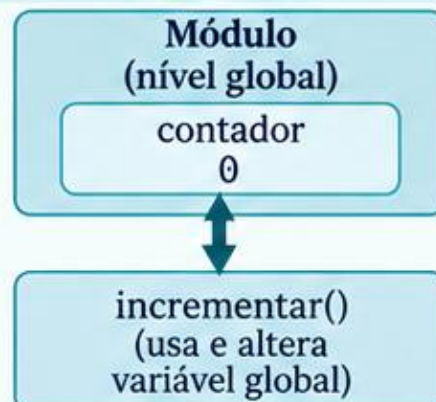
```
def incrementar():  
    global contador  
    contador += 1
```

```
incrementar()  
print(contador)
```

Saída

1

Diagrama



2 Constante (por convenção)

Por convenção, as constantes são escritas em maiúsculas. Python não impõe a imutabilidade das constantes, mas esta convenção ajuda na legibilidade do código.

```
TAXA_DESCONTO = 0.10
```

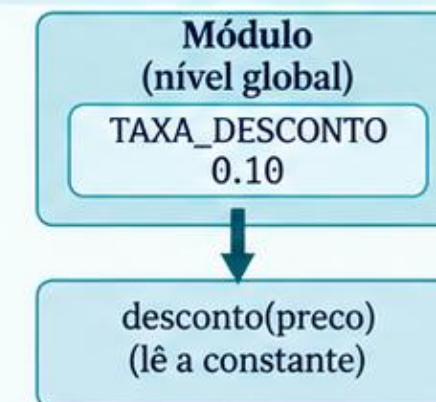
```
def desconto(preco):  
    return preco * TAXA_DESCONTO
```

```
print(desconto(100))
```

Saída

10.0

Diagrama



Variáveis globais: definidas no nível do módulo, acessíveis nas funções. Use **global** para reatribuir o seu valor numa função.



Constantes: por convenção, escritas em maiúsculas. Python não impõe a sua imutabilidade.



Boa prática: prefira parâmetros explícitos e valores de retorno a um estado global excessivo.

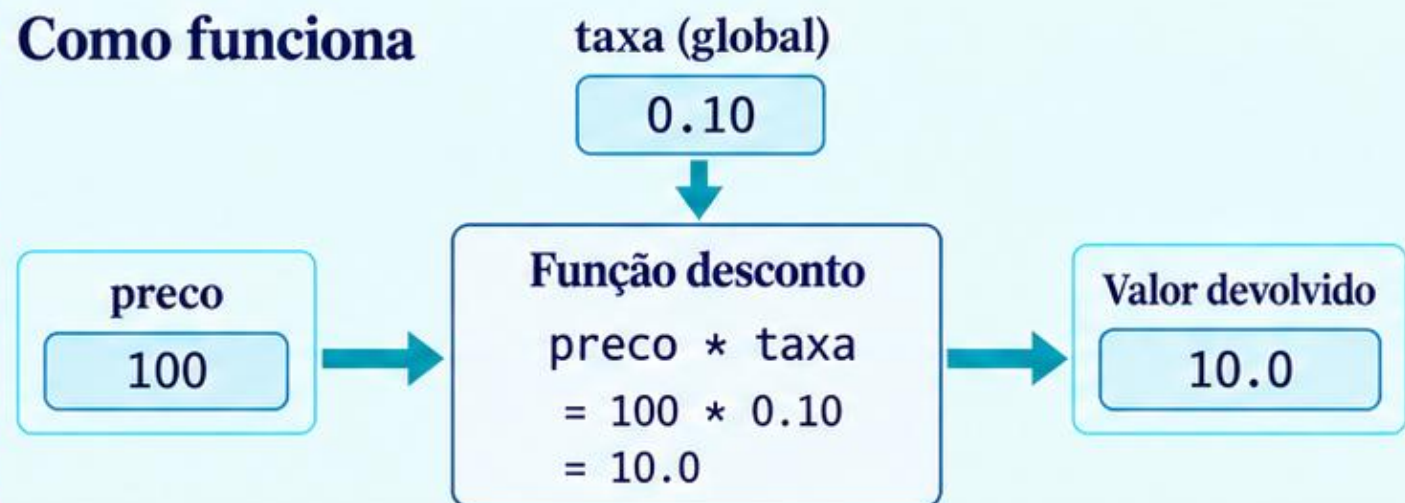
Funções com dependências explícitas

Usa um valor global

```
taxa = 0.10
```

```
def desconto(preco):  
    return preco * taxa
```

Como funciona

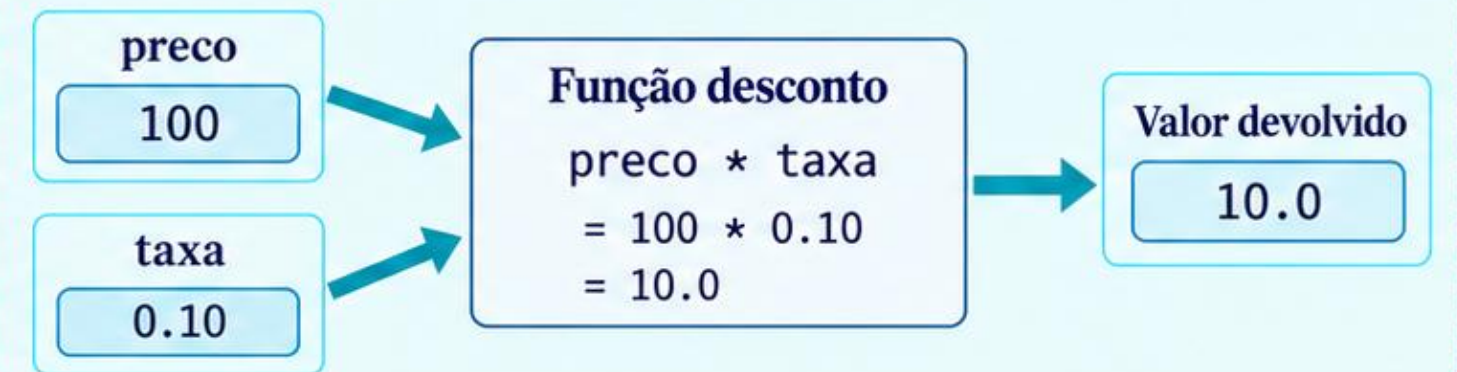


A função lê um valor global, por isso o resultado depende de um estado externo.

Usa parâmetros explícitos

```
def desconto(preco, taxa):  
    return preco * taxa
```

Como funciona



Os valores são fornecidos como parâmetros explícitos.
Por exemplo: `desconto(100, 0.10) → 10.0` e `desconto(100, 0.20) → 20.0`.

Preferir parâmetros e return

Forneça os valores necessários como parâmetros e devolva o resultado com `return`.

Devolver o montante do desconto

Os exemplos devolvem o montante do desconto, não o preço final.

Evitar input e print na função

A função não deve ler dados (`input`) nem imprimir (`print`). Mantenha apenas o cálculo.

Mais fácil de testar

Uma função que depende apenas dos seus parâmetros é mais fácil de testar diretamente.

Composição de funções

Código em Python

```
def dobro(n):  
    return n * 2
```

```
def quadrado(n):  
    return n * n
```

```
resultado = quadrado(dobro(3))  
print(resultado)
```

Código equivalente

```
intermedio = dobro(3)  
resultado = quadrado(intermedio)  
print(resultado)
```

Composição de funções

Na expressão `quadrado(dobro(3))`, o resultado da função *interna* é usado como argumento da função *externa*.

Fluxo de execução



Avaliação da expressão

A chamada interna `dobro(3)` é avaliada primeiro, produzindo 6. Em seguida, esse valor é passado como argumento a `quadrado`, que devolve 36.

Resultado final

A instrução `print(resultado)` mostra o valor devolvido pela composição de funções:

36

Rastreo de chamadas compostas

Definição das funções

```
def dobro(numero):  
    return numero * 2
```

```
def quadrado(numero):  
    return numero * numero
```

Expressões e resultados

Expressão	Resultado
<code>quadrado(dobro(3))</code>	36
<code>dobro(quadrado(3))</code>	18

A ordem das chamadas é importante: `quadrado(dobro(3))` e `dobro(quadrado(3))` produzem resultados diferentes.

1. Avaliação de `quadrado(dobro(3))`



2. Avaliação de `dobro(quadrado(3))`



Uma função só mostra um resultado no ecrã se for usada a instrução `print`. Caso contrário, o valor é devolvido, mas não é apresentado.

A chamada interior termina primeiro. O seu resultado torna-se o argumento da chamada exterior.

Função principal e organização

Código em Python

```
def dobro(n):  
    return n * 2  
  
def main():  
    numero = int(input('Número: '))  
    resultado = dobro(numero)  
    print('Dobro:', resultado)  
  
if __name__ == '__main__':  
    main()
```

Função dobro:

recebe um valor n e devolve o seu dobro, calculado como $n * 2$.

Função main:

coordena a interação do programa com o utilizador: lê um número, chama a função dobro, recebe o resultado e apresenta-o no ecrã.

Bloco de proteção:

if `__name__ == '__main__':` chama a função `main()` apenas quando o ficheiro é executado diretamente, e não quando é importado noutro programa.

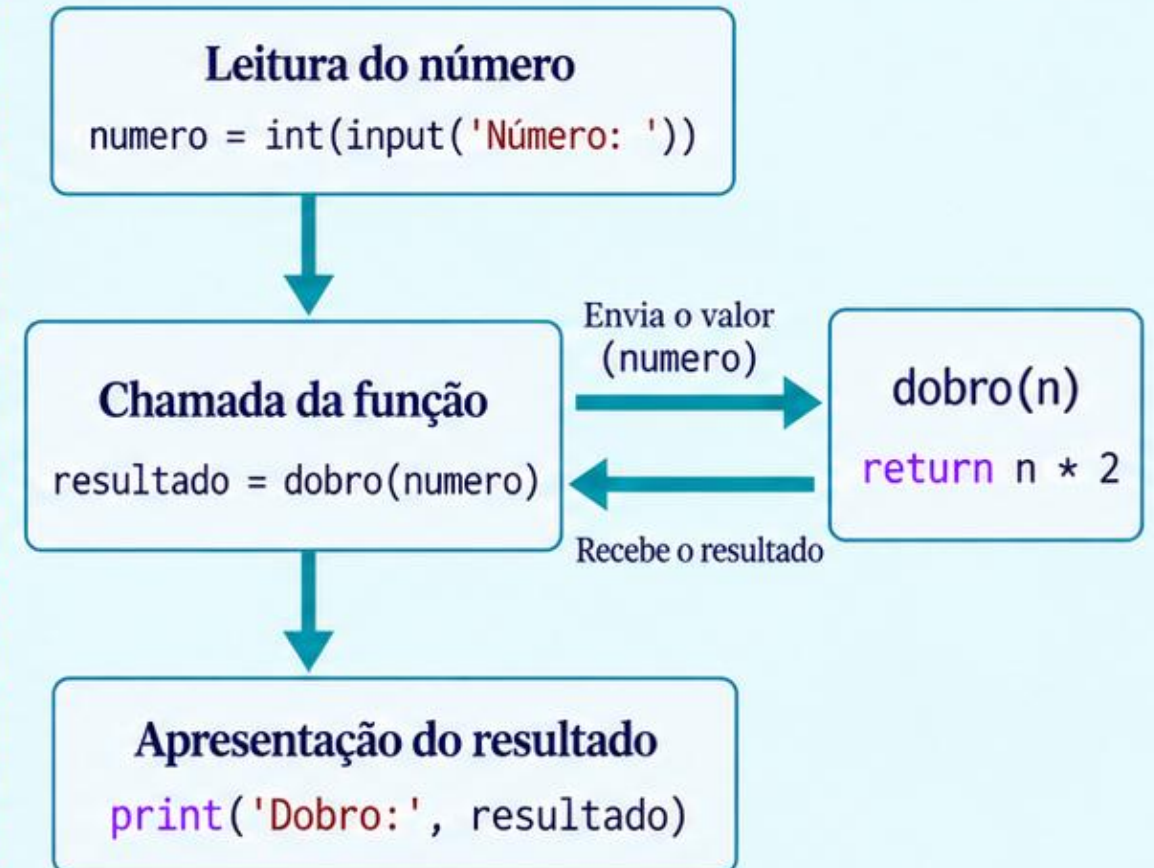
Convenção de nome:

o nome `main` é uma convenção amplamente usada para a função principal de um programa. Não tem um significado especial na linguagem.

Execução e importação:

o bloco `if __name__ == '__main__':` garante que a função `main()` é executada apenas quando o ficheiro é executado diretamente. Se o ficheiro for importado noutro programa, a função `main()` não é chamada.

Funcionamento da função main



Exemplo de execução

Número: 5
Dobro: 10

Módulos próprios

Um módulo próprio é um ficheiro Python com funções que podem ser reutilizadas em outros programas.

Para utilizar as funções de um módulo, importa-se o módulo e acede-se às funções através do nome do módulo.



calculos.py

```
def dobro(numero):  
    return numero * 2  
  
def quadrado(numero):  
    return numero ** 2
```

O ficheiro **calculos.py** define duas funções que podem ser reutilizadas noutros programas.

import calculos

Os ficheiros estão
no mesmo diretório.



calculos.py



programa.py



programa.py

```
import calculos  
  
print(calculos.dobro(5))  
print(calculos.quadrado(5))
```

O programa importa o módulo **calculos** e utiliza as suas funções.

Saída do programa

```
10  
25
```

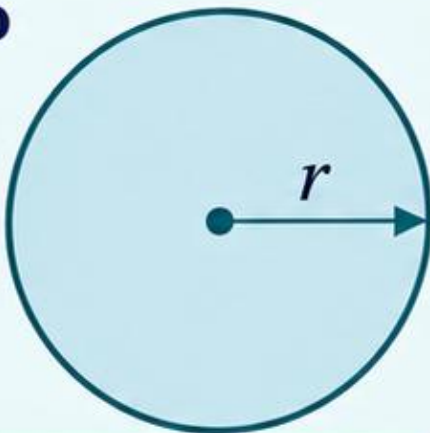
FUNÇÕES MATEMÁTICAS

O módulo `math` da biblioteca padrão de Python fornece funções matemáticas úteis.

Exemplo: área de um círculo

A área de um círculo de raio $r \geq 0$ é dada por:

$$A = \pi r^2$$



```
import math

def area_circulo(raio):
    return math.pi * raio ** 2
```

Expressão	Resultado	Descrição
<code>math.sqrt(25)</code>	5.0	Raiz quadrada de 25. A função <code>sqrt</code> espera um número real não negativo x .
<code>math.pow(2, 3)</code>	8.0	2 elevado a 3.
<code>2**3</code>	8	2 elevado a 3 (operador <code>**</code>).
<code>math.ceil(4.3)</code>	5	Menor inteiro maior ou igual a 4.3.
<code>math.floor(4.3)</code>	4	Maior inteiro menor ou igual a 4.3.
<code>math.ceil(-4.3)</code>	-4	Menor inteiro maior ou igual a -4.3.
<code>math.floor(-4.3)</code>	-5	Maior inteiro menor ou igual a -4.3.
<code>math.pi</code>	3.14159	Aproximação de π (constante <code>pi</code>).

1

Raiz quadrada

A função `math.sqrt(x)` devolve a raiz quadrada de um número real não negativo x (equivalente a $\sqrt{x}$).

2

Funções `ceil` e `floor`

`math.ceil(x)` devolve o menor inteiro maior ou igual a x .
`math.floor(x)` devolve o maior inteiro menor ou igual a x .

3

Área do círculo

A função `area_circulo(raio)` calcula a área πr^2 de um círculo, assumindo $\text{raio} \geq 0$.

Bibliotecas e números aleatórios

Importação de módulos

O módulo random fornece funções para gerar números aleatórios.

Exemplo de utilização

```
import random

dado = random.randint(1, 6)
print(dado)
```

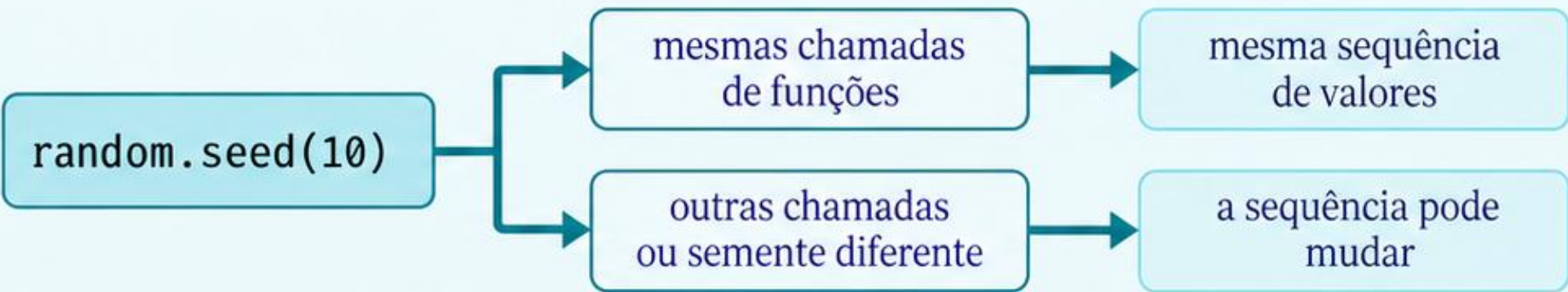
Possível resultado

4

Função	Descrição	Valores possíveis	Notas
random.randint(1, 6)	Inteiro aleatório num intervalo.	1, 2, 3, 4, 5, 6	Inclui ambos os extremos.
random.randrange(2, 11, 2)	Inteiro aleatório com passo.	2, 4, 6, 8, 10	Exclui o valor 11.
random.random()	Número real aleatório.	$0 \leq x < 1$	Valor do tipo float.
random.uniform(10, 20)	Número real aleatório.	Entre 10 e 20	Valor do tipo float.
random.seed(10)	Define a semente do gerador.	—	Permite obter sequências reprodutíveis.

Sementes e reprodutibilidade

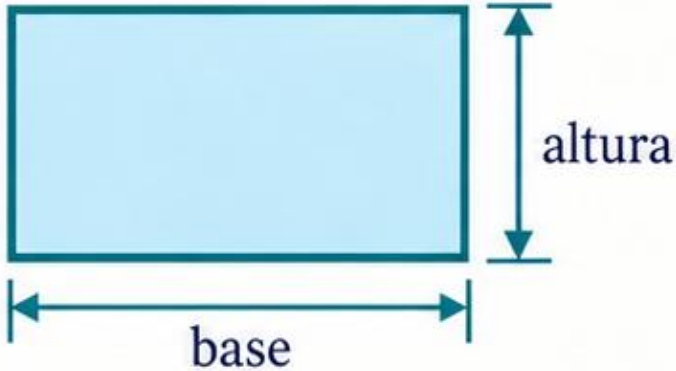
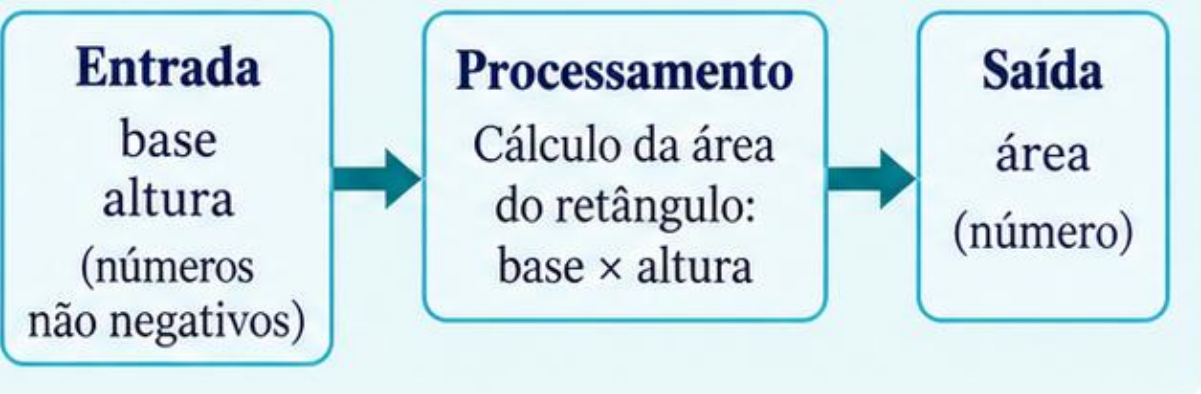
Com a mesma semente e o mesmo conjunto de chamadas, o gerador produz a mesma sequência de números pseudoaleatórios.



CONTRATO DA FUNÇÃO E TESTES

Contrato da função

Define o que a função faz, quais os seus dados de entrada, o processamento, o valor devolvido e eventuais condições.



Implementação

A função segue o contrato definido.

```
def calcular_area(base, altura):  
    """Devolve a área de um retângulo,  
    dada a base e a altura (não negativas)."""  
    return base * altura
```

Testes da função

Cada teste verifica uma responsabilidade da função.

base	altura	área (devolvida)	Descrição do teste
5	10	50	Dimensões positivas (teste típico).
0	10	0	Base nula (área nula).
2.5	4	10.0	Dimensões decimais (números reais).



Observação: O contrato assume que base e altura são não negativas.
A função não valida os dados de entrada. Os testes verificam separadamente os diferentes casos.

Testes automáticos com assert

Código em Python

```
def quadrado(n):  
    return n * n  
  
assert quadrado(3) == 9  
assert quadrado(0) == 0  
assert quadrado(-2) == 4
```

O que é assert?

A instrução `assert` verifica se uma condição é verdadeira. Se for verdadeira, não produz qualquer saída e a execução continua normalmente.

Quando falha?

Se a condição for falsa, é lançada uma exceção `AssertionError` durante a execução normal do programa.

Chamada	Condição verificada	Resultado
<code>quadrado(3)</code>	<code>9 == 9</code>	Passa
<code>quadrado(0)</code>	<code>0 == 0</code>	Passa
<code>quadrado(-2)</code>	<code>4 == 4</code>	Passa

Utilização com valores de ponto flutuante

```
from math import isclose  
assert isclose(0.1 + 0.2, 0.3)
```

Para valores de ponto flutuante, utilize `isclose` para comparar valores com tolerância.

O que acontece quando um teste passa

```
assert  
quadrado(3) == 9
```



Nenhuma saída
e a execução
continua

Não utilizar para validação de entrada

As instruções `assert` não devem ser utilizadas para validar dados fornecidos pelo utilizador. Servem para detetar erros de programação e verificar condições durante o desenvolvimento, não para tratar entradas de utilizadores.

Casos normais e valores-limite

Código em Python

```
def nota_valida(nota):  
    return 0 <= nota <= 20
```

Função:

recebe um argumento numérico e devolve **True** se a nota estiver entre 0 e 20 (inclusive), e **False** caso contrário.

Nota	Valor devolvido	Situação
-1	False	Fora do intervalo
0	True	Limite aceite
10	True	Caso normal
20	True	Limite aceite
21	False	Fora do intervalo

Chamada da função

nota_valida(20)

Argumento
(nota)

Função

```
def nota_valida(nota):  
    return 0 <= nota <= 20
```

Valor devolvido
(True)

Valor devolvido

True

Casos normais

Notas entre 0 e 20 (inclusive), como 0, 10 e 20, são aceites e a função devolve **True**.

Valores-limite

Os valores 0 e 20 são limites aceites e a função devolve **True**.

Fora do intervalo

Notas inferiores a 0 ou superiores a 20, como -1 e 21, estão fora do intervalo e a função devolve **False**.

Testes

```
assert nota_valida(0) is True  
assert nota_valida(20) is True
```

```
assert nota_valida(-1) is False  
assert nota_valida(21) is False
```

A função não inclui qualquer conversão ou validação do tipo do argumento.

Diagnóstico de uma função incorreta

Função incorreta

```
def media(a, b):  
    return a + b / 2
```

O que acontece?

Na expressão $a + b / 2$, a divisão tem precedência sobre a adição. Assim, apenas b é dividido por 2 e, em seguida, o resultado é somado a a .

Chamada	Valor observado	Valor esperado
media(10, 14)	17.0	12.0

Função corrigida

```
def media(a, b):  
    return (a + b) / 2
```

Correção

Para obter a média, é necessário dividir a soma dos dois valores por 2. Usam-se parênteses para garantir a ordem correta de avaliação.

Teste da função

```
assert media(10, 14) == 12.0
```

Passos para diagnosticar e corrigir



Coordenação e menus

Código

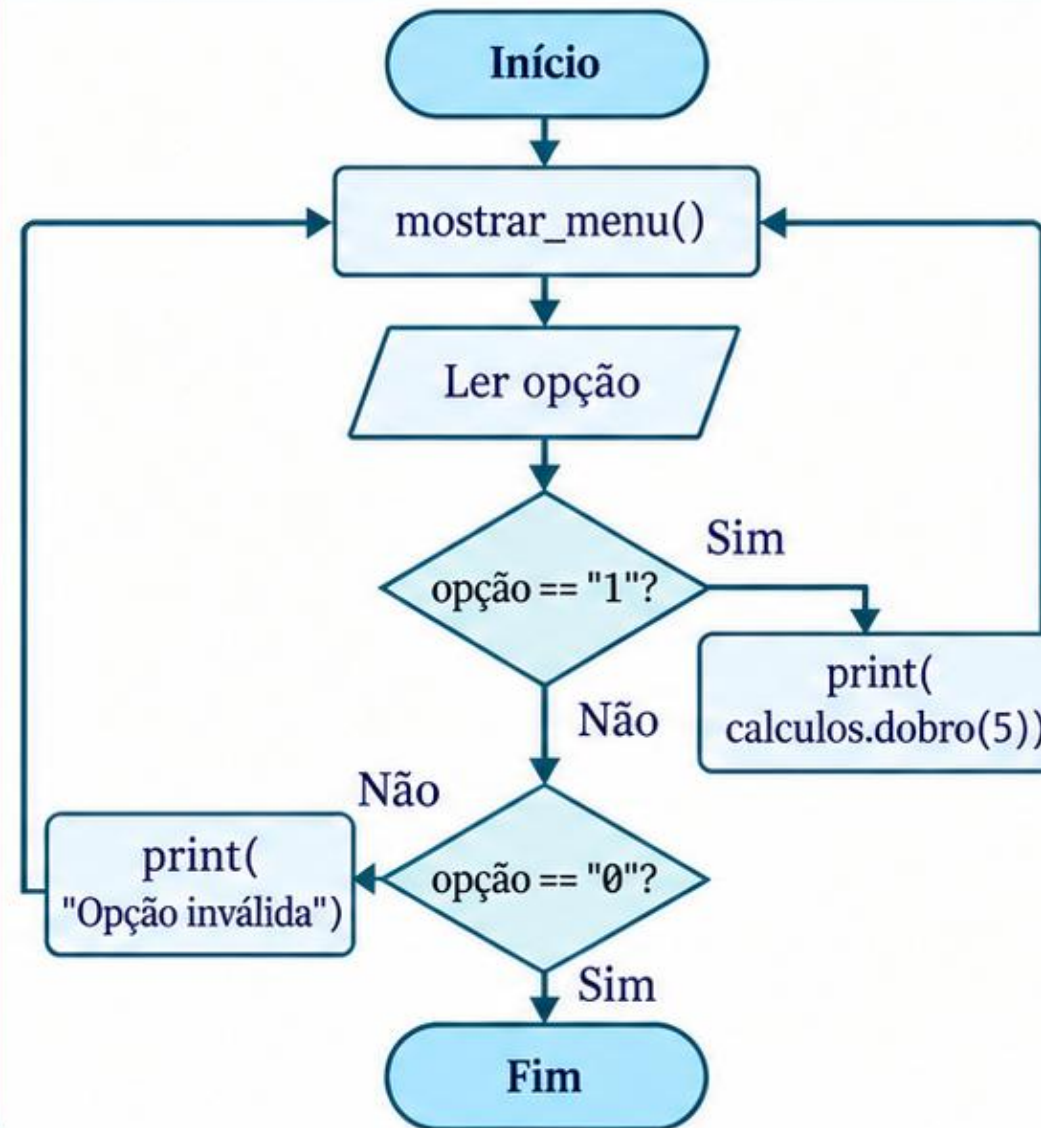
```
import calculos

def mostrar_menu():
    print("1 - Dobro de 5")
    print("0 - Sair")

def main():
    opcao = ""
    while opcao != "0":
        mostrar_menu()
        opcao = input("Opção: ")
        if opcao == "1":
            print(calculos.dobro(5))
        elif opcao != "0":
            print("Opção inválida")

main()
```

Fluxo de execução



Papéis das funções

- 1 **mostrar_menu**: apresenta as opções ao utilizador.
- 2 **main**: controla a repetição e a decisão.
- 3 **calculos.dobro**: realiza o cálculo do dobro.

Exemplos de execução

Entrada	Resultado
1	10
9	Opção inválida
0	Termina o programa

Um menu simples coordena a interação com o utilizador e utiliza funções de um módulo externo.

Reorganização sem alterar resultados

Antes da reorganização

```
preco = 100
taxa = 0.20
total = preco * (1 - taxa)
print(total)
```

Saída

80.0

Depois da reorganização

```
def preco_final(preco, taxa):
    return preco * (1 - taxa)

total = preco_final(100, 0.20)
print(total)
```

Saída

80.0

O que foi feito

A reorganização mantém o mesmo comportamento, mas move o cálculo para uma função reutilizável.

Contratos assumidos

Considera-se que:

- `preco >= 0`
- `0 <= taxa <= 1`

Como funciona a nova versão

Argumentos

100, 0.20

Função

```
preco_final(preco, taxa)
preco * (1 - taxa)
```

Valor devolvido

80.0

Resultados de teste

Chamada	Valor recebido	Valor devolvido
<code>preco_final(100, 0.20)</code>	100, 0.20	80.0
<code>preco_final(100, 0)</code>	100, 0	100
<code>preco_final(0, 0.20)</code>	0, 0.20	0.0

Vantagens

Melhora a legibilidade e facilita os testes, permitindo usar o mesmo cálculo com diferentes valores, sem alterar o resultado.

Exercício: programa de classificação

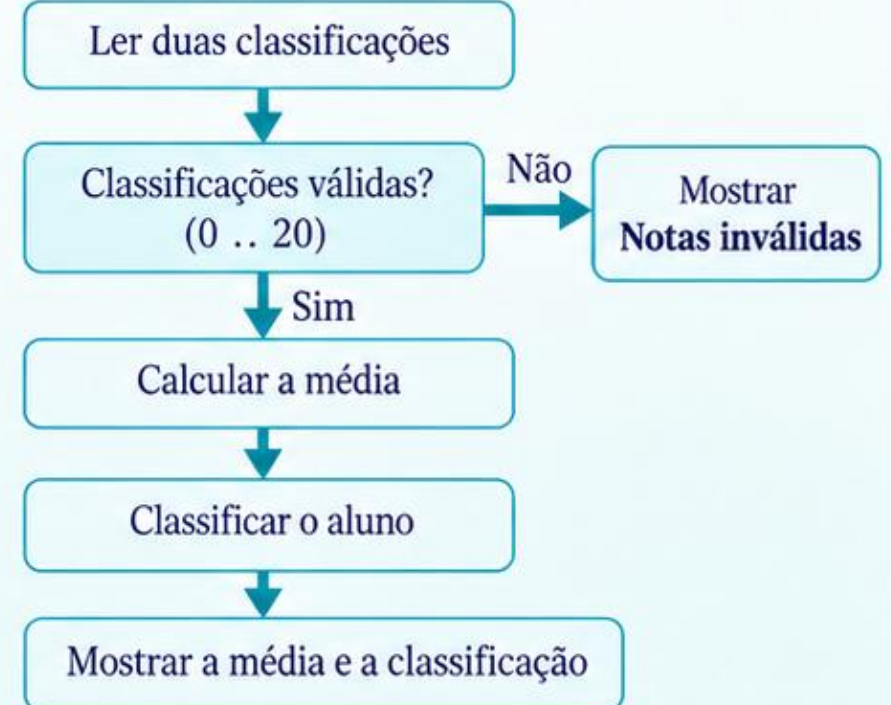
Objetivo do programa

Ler duas classificações numéricas, verificar se são válidas (no intervalo 0..20) e, se forem válidas, calcular a média aritmética e classificar o aluno. Caso contrário, mostrar a mensagem **Notas inválidas**.

Funções do programa

<code>nota_valida(n)</code>	verifica se uma classificação está no intervalo 0..20 e devolve um valor lógico.
<code>media(a, b)</code>	calcula a média aritmética de duas classificações e devolve o valor da média.
<code>classificar(n)</code>	devolve a classificação do aluno: "Aprovado" se $n \geq 10$, caso contrário "Reprovado".
<code>main()</code>	lê as classificações, utiliza as funções e trata da entrada e saída de dados.

Fluxo de funcionamento



Testes do programa

Classificação 1	Classificação 2	Média	Resultado
8	12	10.0	Aprovado
8	10	9.0	Reprovado
-1	12	—	Notas inválidas
20	20	20.0	Aprovado

Valores devolvidos pelas funções

`nota_valida(n)`
Devolve **True** se a classificação estiver no intervalo 0..20; caso contrário, devolve **False**.

`media(a, b)`
Devolve o valor da média aritmética das duas classificações.

`classificar(n)`
Devolve a cadeia de caracteres "Aprovado" se $n \geq 10$; caso contrário, devolve "Reprovado".

A função `main()` lê as classificações, utiliza as funções e mostra os resultados. Assume-se que a conversão para valor numérico é realizada corretamente.

Solução: funções e coordenação

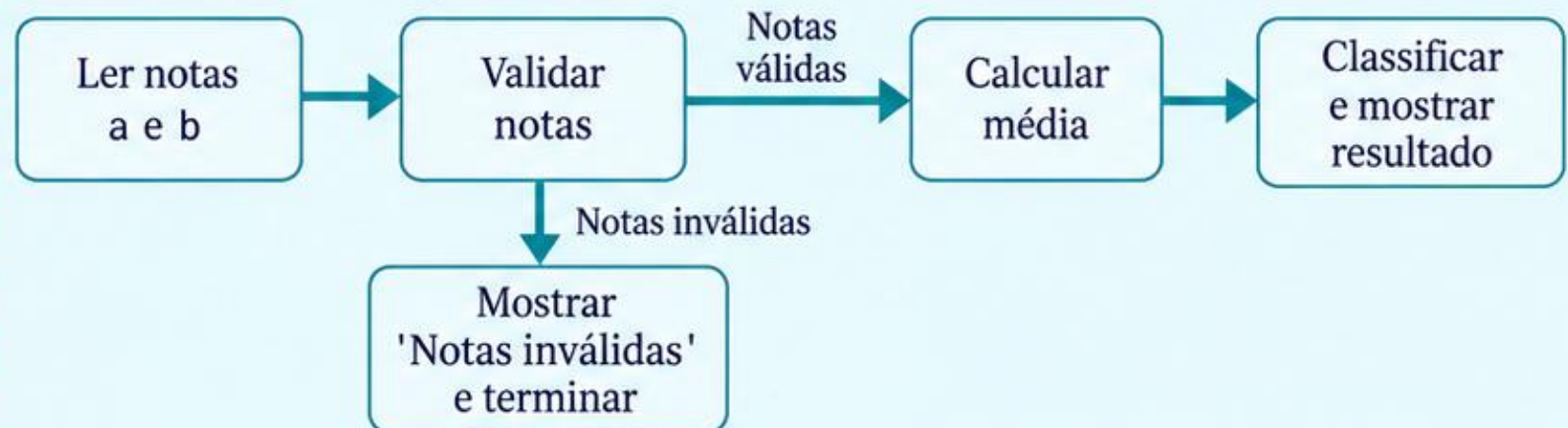
Código em Python

```
def nota_valida(n):  
    return 0 <= n <= 20  
  
def media(a, b):  
    return (a + b) / 2  
  
def classificar(n):  
    if n >= 10:  
        return 'Aprovado'  
    return 'Reprovado'  
  
def main():  
    a = float(input('Nota 1: '))  
    b = float(input('Nota 2: '))  
    if not (nota_valida(a) and nota_valida(b)):  
        print('Notas inválidas')  
        return  
    m = media(a, b)  
    print('Média:', m)  
    print(classificar(m))  
  
main()
```

Funções e responsabilidades

Função	Responsabilidade
nota_valida(n)	Validar se uma nota está entre 0 e 20.
media(a, b)	Calcular a média de duas notas.
classificar(n)	Classificar a nota como 'Aprovado' ou 'Reprovado'.
main()	Coordenar a leitura das notas, validar, calcular, mostrar a média e a classificação.

Fluxo de execução em main



Retorno antecipado

Se alguma das notas não for válida, a função main mostra a mensagem 'Notas inválidas' e termina imediatamente, usando **return**. Assim, as restantes instruções não são executadas.

Testes das funções e do programa

Testes das funções (testes unitários)

```
assert nota_valida(-1) is False
assert nota_valida(20) is True
assert media(8, 12) == 10.0
assert classificar(9.9) == 'Reprovado'
assert classificar(10) == 'Aprovado'
```

Testes das funções:

verificam o comportamento de cada função de forma isolada, usando instruções `assert`. Confirmam que as funções devolvem os valores esperados para diferentes entradas.

Execução do programa:

testa o programa completo, incluindo a entrada e a saída de dados. A tabela mostra alguns exemplos de execução com dois inputs e a respetiva saída.

Exemplos de execução do programa (dois inputs e saída)

Notas de entrada	Média	Classificação (saída)
(8, 12)	10.0	Aprovado
(8, 10)	9.0	Reprovado
(-1, 12)	—	Notas inválidas
(20, 20)	20.0	Aprovado

Dois tipos de testes:

os testes **unitários** validam cada função individualmente; a execução do programa confirma o comportamento global, com entrada e saída de dados.

Funções utilizadas:

`nota_valida`, `media` e `classificar` (definidas no slide anterior).

Síntese – Funções II

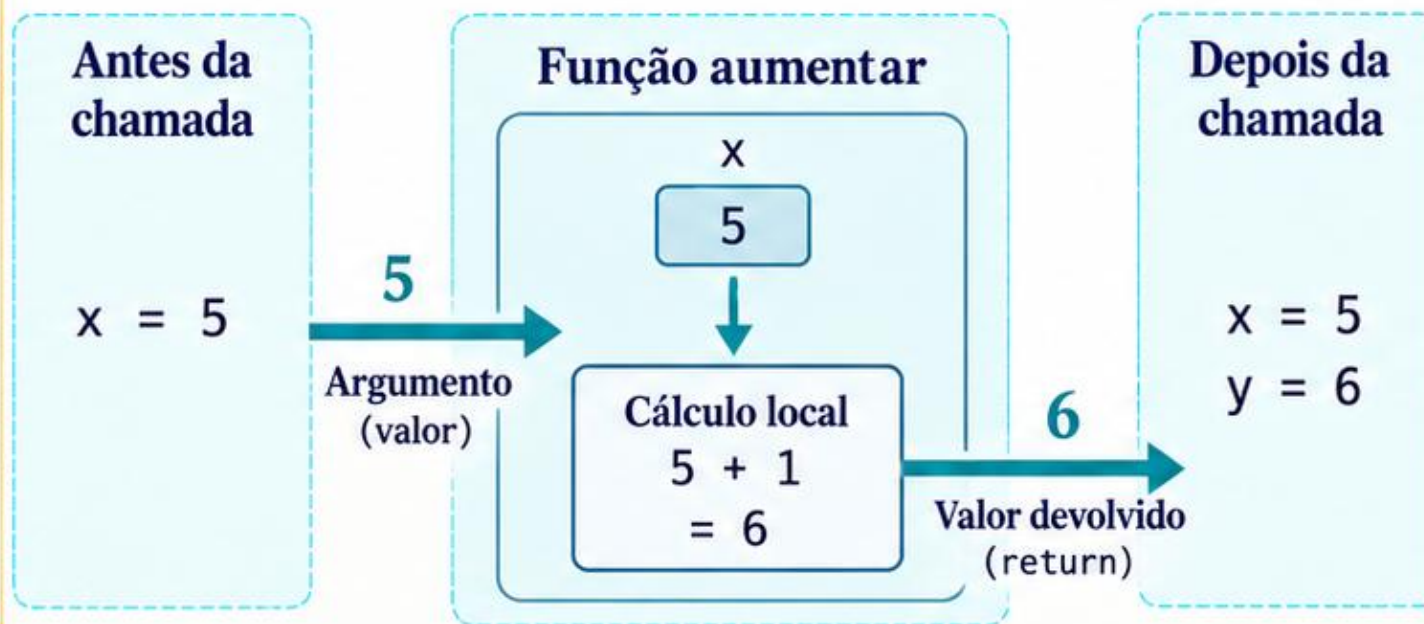
Exemplo rápido

`x = 5`

```
def aumentar(x):  
    return x + 1
```

```
y = aumentar(x)  
print(x, y)
```

Como funciona



Saída no ecrã

5 6

Explicação

O parâmetro `x` da função é uma variável local e não reatribui o `x` global. O valor **6** devolvido pela função é atribuído a `y`.

1. Nomes locais

As variáveis definidas dentro da função (como `x`) são locais à função.

2. Parâmetros

Os parâmetros tornam explícitas as entradas necessárias para a função.

3. Return

A instrução `return` permite obter um valor da função, que pode ser usado noutras expressões.

4. Função principal

Uma função principal (orquestra) coordena a execução do programa.

5. Módulos

Os módulos permitem reutilizar funções em diferentes programas.

6. Testes

É importante fazer testes antes e depois da reorganização do código para verificar se o comportamento se mantém.

Verificação final

Consegue identificar as entradas, os resultados, as responsabilidades e os testes?